



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA



Fundamentos de Segurança Informática

Trabalho Prático 3

Relatório

Data: 31/05/2026

Trabalho realizado no âmbito da edição de 2025/2026 da disciplina de Fundamentos de Segurança Informática, da responsabilidade do docentes António Jorge da Costa Granjal, Rogério Luís de Carvalho Costa do Departamento de Engenharia Informática, da Licenciatura em Engenharia Informática da Faculdade de Ciências e Tecnologias da Universidade de Coimbra pelos alunos Luís Filipe Cardetas Banha – nº2023217305 e Rui Mário Teixeira Camarão – nº2023216522 da turma PL5.

1. Introdução

O objetivo principal deste trabalho prático é explorar a segurança de aplicações web e implementar uma *web application firewall* (WAF) para proteger a aplicação contra ataques na camada aplicacional. A aplicação alvo a ser utilizada neste trabalho é o OWASP Juice Shop

Para atingir este objetivo, o trabalho está dividido em duas fases:

- **Fase 1:** Foca-se em explorar a segurança da aplicação web Juice Shop aplicando as diretrizes do guia de testes de segurança web da OWASP (WSTG - *Web Security Testing Guide*) para identificar vulnerabilidades.
- **Fase 2:** Tem como objetivo monitorizar, filtrar e bloquear o tráfego HTTP direcionado à Juice Shop, através da implementação de uma WAF utilizando o ModSecurity, com o intuito de mitigar as vulnerabilidades identificadas na primeira fase.

2) Arquitetura considerada para o Trabalho Prático 3

A arquitetura para este trabalho varia consoante a fase de testes, transitando de um modelo de ataque direto para um modelo com tráfego filtrado.

Cenário 1: Testes de segurança web (*Web security testing*)

Este cenário representa a fase inicial de avaliação de vulnerabilidades, onde não existem medidas defensivas implementadas.

- **Estrutura da Rede:** O cliente de testes de penetração estabelece uma linha de comunicação direta com o servidor web que aloja a aplicação.
- **Servidores:**
 - **Servidor Cliente:** A máquina do atacante, tipicamente equipada com ferramentas de segurança como o OWASP ZAP ou outras disponíveis no Kali Linux.
 - **Servidor Web:** A máquina que executa a aplicação alvo.
- **Serviços:** O serviço alvo é a aplicação OWASP Juice Shop (sendo recomendada a versão mais recente, como a v17.2.0).

Cenário 2: Web application firewall

Este cenário introduz a camada defensiva para proteger a aplicação com base no relatório da primeira fase.

- **Estrutura da Rede:** Uma WAF é implementada entre o cliente de testes de penetração e o servidor web. O objetivo é que a WAF intercete, monitorize e filtre todo o tráfego HTTP antes de este chegar ao servidor web.
- **Servidores:**
 - **Servidor Cliente:** A máquina de testes (Kali Linux / OWASP ZAP).
 - **VM Servidor (WAF + Web Server):** Uma única máquina que aloja simultaneamente o serviço de *firewall* e a aplicação. A aplicação é configurada para aceitar apenas ligações locais, garantindo que o tráfego externo não a consegue contornar.
- **Serviços:** A WAF de fronteira é composta por um serviço Apache 2 integrado com o ModSecurity (utilizando o OWASP CRS). Este serviço Apache atua como *reverse proxy*, recebendo os pedidos HTTP externos e encaminhando-os internamente (ex: via *localhost*) para a aplicação OWASP Juice Shop.

3) Web application security testing (Testes de segurança da aplicação web)

Esta secção detalha os testes de penetração realizados contra a aplicação OWASP Juice Shop, seguindo as diretrizes do *Web Security Testing Guide* (WSTG) da OWASP. As ferramentas principais utilizadas para esta análise incluíram o OWASP ZAP, executando mapeamentos automáticos, varrimentos ativos e testes manuais.

1 - Information Gathering

4.1.1 Conduct Search Engine Discovery Reconnaissance for Information Leakage: Esta secção está out-of-scope, pois a aplicação juiceshop está a ser executada em um ambiente isolado e não está indexado em nenhum motor de busca público.

4.1.2 Fingerprint Web Server: Para identificar o servidor web em uso, foi adotada uma abordagem ativa utilizando a ferramenta Nmap (nativa do Kali Linux). O objetivo foi realizar um varrimento de versão focado no porto do serviço web.

Comando utilizado: `nmap -sV -p 3000 10.0.0.1(ip do servidor)`

Descobertas: Embora o Nmap não tenha conseguido identificar o serviço de forma limpa através da sua base de dados de assinaturas padrão (retornando ppp?), a recolha do fingerprint bruto (1 service unrecognized despite returning data) revelou-se extremamente útil. A análise manual do output do Nmap expôs a resposta HTTP completa do servidor.

Análise de Cabeçalhos e Conteúdo: A leitura do fingerprint permitiu identificar cabeçalhos HTTP cruciais (como HTTP/1.1 200 OK e Access-Control-Allow-Origin: *) e o próprio código-fonte da página inicial. Destacam-se as tags HTML que confirmam a identidade da aplicação: <title>OWASP Juice Shop</title> e os direitos de autor <meta name="description" content="Probably the most modern and sophisticated insecure web application">.

[illegible]

4.1.3 Review Webserver Metafiles for Information Leakage:

Através do ZAP, procurou-se por ficheiros de metadados padrão, como o robots.txt ou o security.txt, para identificar caminhos ocultos ou informações de segurança.

- **Descobertas:** A consulta a estes ficheiros na raiz do servidor web revelou-se extremamente útil, pois, embora o seu propósito original seja orientar motores de busca ou fornecer dados de contacto, acabaram por expor a estrutura interna da aplicação a potenciais atacantes.

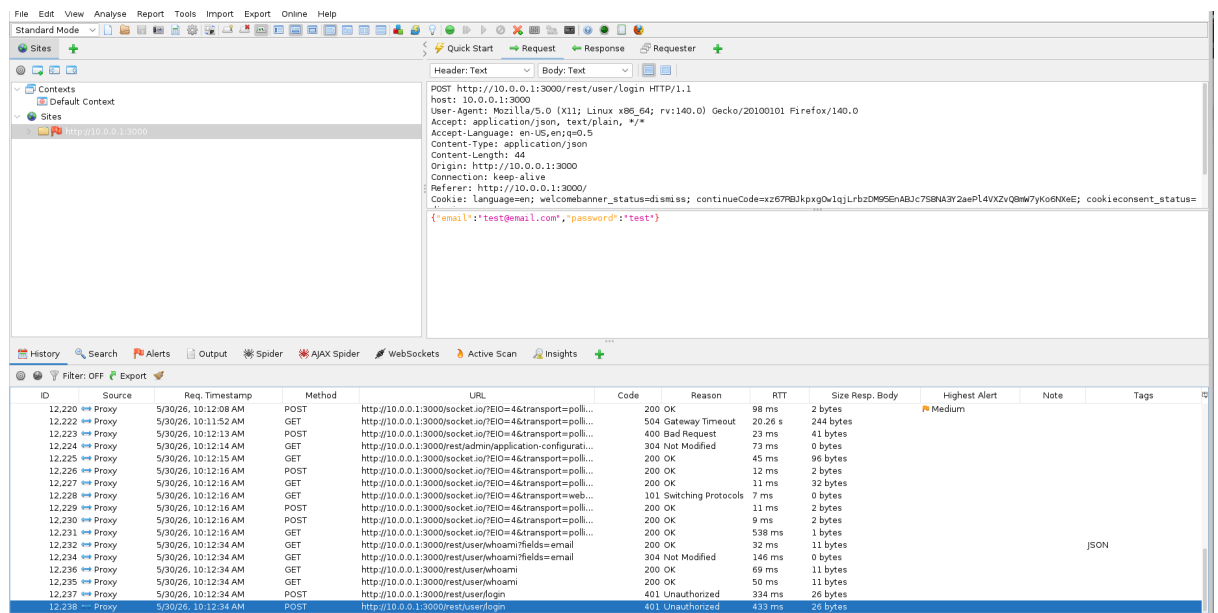
- **Análise de Ficheiros e Conteúdo:** A leitura do ficheiro robots.txt permitiu identificar a diretiva Disallow: /ftp, revelando de forma inadvertida a existência de um diretório oculto e potencialmente sensível (Information Leakage). Adicionalmente, a análise do ficheiro de políticas de segurança (security.txt ou .well-known/security.txt) expôs vários endereços internos que não seriam imediatamente óbvios, destacando-se os caminhos Acknowledgements: /#/score-board (confirmando a localização do painel de pontuações) e Hiring: /#/jobs

4.1.4 Enumerate Applications on Webserver: Embora o WSTG sugira técnicas avançadas (como pesquisas de IP reverso para descobrir domínios partilhados num mesmo servidor), no contexto deste ambiente de laboratório isolado, a utilização do Nmap confirmou que o servidor está dedicado exclusivamente a alojar o porto do serviço web. Não existem outras aplicações partilhando o mesmo host.

4.1.5 Review Webpage Content for Information Leakage: Seguindo as recomendações do guia, combinou-se o varrimento passivo do OWASP ZAP com a inspeção manual utilizando as Ferramentas de Programador do Web Browser (*Developer Tools*). Analisou-se o código-fonte HTML e ficheiros JavaScript em busca de comentários ou outras fugas de informação sensível.

- **Descobertas:** O varrimento automático do ZAP detetou com sucesso múltiplas instâncias de fuga de informação (*Information Leakage*) nas respostas da aplicação. Destacam-se os seguintes alertas confirmados:
 - **Information Disclosure - Suspicious Comments:** O ZAP identificou comentários no código-fonte que revelam detalhes de implementação interna.
 - **Private IP Disclosure:** Foi detetada a exposição de endereços IP privados/internos no conteúdo das páginas.
 - **Timestamp Disclosure - Unix:** Assinalou-se a divulgação de *timestamps* no formato Unix, úteis para determinar informações temporais do sistema. (*Insere aqui o teu screenshot dos alertas do ZAP*).

4.1.6 Identify Application Entry Points: A identificação foi realizada através da navegação manual com interceção de tráfego ativa no proxy do OWASP ZAP. Mapearam-se os principais pontos de entrada de dados, o que incluiu métodos GET/POST em formulários de login, barras de pesquisa, cabeçalhos HTTP manipuláveis e parâmetros de URL.



4.1.7 Map Execution Paths Through Application: Para mapear e rastrear os caminhos de execução da aplicação, utilizaram-se as ferramentas *Spider* e, fundamentalmente, o *AJAX Spider* do OWASP ZAP.

- **Descobertas:** Como a aplicação alvo é rica em JavaScript, o *Spider* tradicional revelou-se insuficiente por si só. A utilização do *AJAX Spider* permitiu explorar a aplicação renderizando o JavaScript através de um *browser* embutido, descobrindo novos caminhos (URLs) e *endpoints* da API que não estavam visíveis estaticamente. Adicionalmente, esta fase foi complementada com ferramentas de descoberta de diretórios para localizar caminhos não referenciados.

○

4.1.8 Fingerprint Web Application Framework: A identificação das frameworks tecnológicas baseou-se na análise dos pedidos HTTP intercetados no ZAP, inspecionando especificamente cabeçalhos e a estrutura do código-fonte HTML. Complementou-se esta análise com ferramentas de *fingerprinting* passivo, como a extensão de browser Wappalyzer.

- **Descobertas:** A conjugação destas metodologias revelou marcadores específicos no código e nas respostas do servidor que indicam a arquitetura tecnológica subjacente. Confirmou-se que a aplicação é construída utilizando **Node.js** e **Express** no *backend*, e utiliza a framework **Angular** no *frontend*.

4.1.9 Fingerprint Web Application: Embora a versão 4.2 do WSTG indique que o conteúdo desta secção foi fundido com a secção anterior (4.1.8), procedeu-se à verificação da aplicação em si.

- **Descobertas:** O varrimento passivo, aliado aos marcadores visuais, títulos de página (<title>OWASP Juice Shop</title>) e comentários de direitos de autor presentes no código-fonte, confirmou inequivocamente que a aplicação alvo é a versão específica do **OWASP Juice Shop** requerida para o projeto.

4.1.10 Map Application Architecture: O mapeamento da arquitetura tem como objetivo determinar os diferentes componentes utilizados para construir a aplicação web. A interação inicial, desenhada e monitorizada através da árvore de sites (*Sites Tree*) do OWASP ZAP, permitiu compreender o fluxo de dados.

- **Descobertas:** Confirmou-se que a arquitetura assenta num modelo de *Single Page Application* (SPA). Em termos práticos, isto significa que o cliente carrega a interface principal numa primeira instância e, posteriormente, comunica de forma assíncrona (via AJAX/XHR) com uma API RESTful dedicada no *backend* para executar operações de dados.

2 - Configuration and Deployment Management Testing

4.2.1 Test Network Infrastructure Configuration e 4.2.2 Test Application

Platform Configuration: A configuração da plataforma aplicacional foi avaliada com recurso aos alertas gerados pelo varrimento automático do OWASP ZAP.

Descobertas: O ZAP detetou a ausência de cabeçalhos de segurança HTTP essenciais para mitigar ataques do lado do cliente:

- **Content Security Policy (CSP) Header Not Set:** A aplicação não define uma política de segurança de conteúdo, o que facilita a injeção e execução de scripts maliciosos (XSS).
- **X-Content-Type-Options Header Missing:** A ausência deste cabeçalho permite que o *browser* tente adivinhar o tipo de conteúdo (MIME sniffing), podendo levar a vulnerabilidades de injeção caso ficheiros de texto sejam interpretados como executáveis ou scripts.

4.2.3 Test File Extensions Handling for Sensitive Information: O objetivo deste teste foi verificar como o servidor web lida com diferentes extensões de ficheiros e determinar se extensões sensíveis (como ficheiros de backup, scripts ou dados em

bruto) são bloqueadas ou indevidamente servidas aos utilizadores. **Descobertas:** Confirmou-se que o servidor web permite o acesso e o descarregamento direto de ficheiros com extensões sensíveis ou incomuns, como .bak (backups) e .md (Markdown). O servidor serve estes ficheiros diretamente em vez de restringir o acesso, o que resulta na exposição de informações internas e credenciais esquecidas.

4.2.4 Review Old Backup and Unreferenced Files for Sensitive Information:

Conforme identificado previamente através da análise do ficheiro robots.txt, procedeu-se à exploração manual de diretórios não referenciados na interface principal. **Descobertas:** O acesso direto ao diretório /ftp confirmou a existência de ficheiros antigos e backups expostos que não possuem qualquer mecanismo de controlo de acesso. A exposição destes ficheiros compromete a segurança ao revelar informações internas e potenciais vulnerabilidades antigas da aplicação.

4.2.5 Enumerate Infrastructure and Application Admin Interfaces: O objetivo deste teste consistiu em identificar interfaces de administração e funcionalidades privilegiadas que pudessem estar ocultas, avaliando se utilizadores não autorizados conseguiriam aceder às mesmas. **Descobertas:** Com base na enumeração de diretórios e na revisão do código-fonte efetuada anteriormente, identificaram-se referências a caminhos administrativos que não se encontram visíveis na interface padrão de um utilizador comum. A existência destas interfaces (como /#/administration) exige testes de autorização rigorosos para garantir que o acesso direto não é permitido a utilizadores sem os devidos privilégios.

4.2.6 Test HTTP Methods: Verificou-se quais os métodos HTTP (verbos) suportados pelo servidor para identificar configurações permissivas. **Descobertas:** Através da inspeção de respostas HTTP intercetadas, confirmou-se que o servidor permite métodos como GET, HEAD, PUT, PATCH, POST e DELETE. A permissividade de métodos como PUT e DELETE deve ser estritamente controlada e limitada a *endpoints* da API que requeiram autenticação.

4.2.7 Test HTTP Strict Transport Security: A funcionalidade *HTTP Strict Transport Security* (HSTS) serve para instruir o *browser* a estabelecer ligações exclusivamente através de canais encriptados (HTTPS), prevenindo ataques de interceção de tráfego. **Descobertas:** A aplicação encontra-se a ser executada localmente através de HTTP. Como não existe configuração de certificados TLS/SSL no servidor web de origem, o cabeçalho HSTS não é enviado nas respostas. Isto confirma a ausência de proteção contra interceção de credenciais em trânsito (*sniffing*), sublinhando a necessidade da futura implementação da WAF e de *Reverse Proxy* com suporte a HTTPS.

4.2.8 Test RIA Cross Domain Policy: Avaliaram-se as políticas que permitem o acesso de domínios cruzados a dados e serviços da aplicação. **Descobertas:** O OWASP ZAP gerou um alerta de **Cross-Domain Misconfiguration (Systemic)**.

Esta má configuração nas políticas de *Cross-Origin Resource Sharing* (CORS) ou ficheiros de política de domínios pode permitir que domínios de terceiros efetuem pedidos não autorizados e acessem a dados sensíveis dos utilizadores da aplicação.

Secções Não Aplicáveis (Out of Scope): De acordo com as diretrizes e instruções explícitas do trabalho prático, as seguintes secções não se aplicam a este cenário e são consideradas fora de âmbito (*out of scope*):

- **4.2.9 Test File Permission:** Não aplicável no ambiente de testes isolado utilizado.
- **4.2.10 Test for Subdomain Takeover:** A aplicação corre em laboratório local, sem configuração de domínios ou subdomínios externos.
- **4.2.11 Test Cloud Storage:** A aplicação não depende de serviços externos de *Cloud Storage* (como Amazon S3) para este cenário.

3- Identity Management Testing

4.3.1 Test Role Definitions: O objetivo deste teste foi verificar se a aplicação impõe uma separação correta de privilégios (*roles*) entre administradores e utilizadores comuns. **Descobertas:** Para validar a segregação de funções, efetuou-se a autenticação com uma conta de cliente padrão e tentou-se aceder diretamente ao painel de administração (`/#/administration`), cujo caminho havia sido descoberto na fase de *Information Gathering*. A aplicação respondeu com um erro **403 Forbidden**, demonstrando que, pelo menos para a renderização deste *endpoint* específico, existe um controlo de acessos baseado em funções (*Role-Based Access Control* - RBAC) que impede utilizadores sem privilégios administrativos de visualizar a página.

4.3.2 Test User Registration Process e 4.3.3 Test Account Provisioning

Process: Estas secções avaliam a segurança do processo de registo e a forma como a infraestrutura atribui (provisiona) novas contas. **Descobertas:** Criou-se uma nova conta utilizando um endereço de correio eletrónico fictício e sem sentido. Verificou-se que a aplicação regista o utilizador, provisiona a conta imediatamente e permite o *login* automático sem exigir qualquer mecanismo de verificação *out-of-band* (como o envio de um *link* de confirmação para o email registado ou aprovação por parte da administração). Esta falha no processo de registo facilita a criação automatizada e massiva de contas falsas, o que pode ser explorado para mascarar ataques, esgotar recursos ou manipular a lógica de negócio (ex: avaliações falsas de produtos).

4.3.4 Testing for Account Enumeration and Guessable User Account: O objetivo deste teste foi determinar se a aplicação permite a um atacante deduzir ou verificar a existência de contas de utilizador válidas no sistema. **Descobertas:** Ao tentar

registar uma nova conta utilizando o endereço de email de um utilizador já existente (admin@juice-sh.op), a aplicação rejeitou o registo devolvendo a mensagem de erro explícita **"Email must be unique"**. Esta resposta diferenciada (em oposição a uma mensagem genérica) confirma inequivocamente ao atacante que o email submetido já se encontra registado na base de dados. Esta é uma vulnerabilidade clássica de Enumeração de Contas (*Account Enumeration*), que facilita significativamente ataques direcionados de força bruta (*brute-force*), pois o atacante tem a certeza de quais são as credenciais válidas a atacar.

The screenshot shows a web browser window with several tabs open: Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, Exploit-DB, and Google Hacking DB. The browser address bar shows 'op'. The page has a dark theme. The 'User Registration' form is centered and contains the following elements:

- Title:** User Registration
- Email:** Input field with 'admin@juice-sh.op' and a red error message 'Email must be unique'.
- Password:** Input field with dots, a message 'Password must be 5-40 characters long. 8/20', and a 'Show password advice' link.
- Repeat Password:** Input field with dots and a message '8/40'.
- Security Question:** Dropdown menu with 'Your favorite movie?' and a message 'This cannot be changed later!'.
- Answer:** Input field with 'test'.
- Register Button:** A button with a user icon and the text 'Register'.
- Link:** 'Already a customer?' at the bottom right.

4.3.5 Testing for Weak or Unenforced Username Policy: Avaliou-se se a aplicação impõe políticas rigorosas na escolha do nome de utilizador (neste caso, o campo de *email* usado como identificador principal). **Descobertas:** Os testes efetuados no processo de registo demonstraram que a aplicação aceita endereços completamente inválidos ou com formatos arbitrários. A não aplicação de expressões regulares (*Regex*) estritas ou a falta de validação real da estrutura da identidade fornecida evidencia uma política de nomes de utilizador fraca. Isto degrada a fiabilidade da base de dados e dificulta a monitorização e auditoria de utilizadores legítimos.

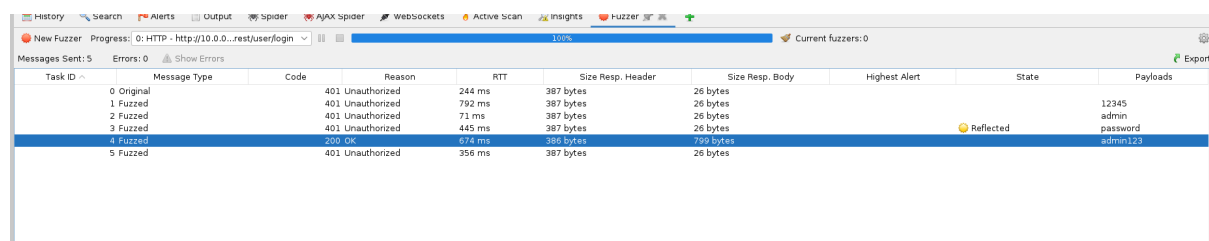
4 - Authentication Testing

4.4.1 Testing for Credentials Transported over an Encrypted Channel: O objetivo deste teste é garantir que as credenciais de autenticação são transmitidas de forma segura pela rede. **Descobertas:** Como a aplicação está a ser executada

num ambiente de laboratório local (porta 3000) sem a configuração prévia de um certificado TLS/SSL, toda a comunicação é feita através do protocolo HTTP não encriptado. Consequentemente, a intercepção do tráfego no OWASP ZAP permitiu confirmar que as credenciais de *login* (email e palavra-passe) são transmitidas em texto limpo (*plaintext*), estando vulneráveis a ataques de intercepção (*sniffing* / *Man-in-the-Middle*).

4.4.2 Testing for Default Credentials: O objetivo deste teste é verificar se a aplicação ou a sua infraestrutura foram implementadas mantendo credenciais de fábrica ou contas padrão. **Descobertas:** Durante os testes manuais e o varrimento do ZAP, verificou-se que a aplicação não depende de credenciais de fabricante tradicionais. No entanto, o ambiente possui contas pré-configuradas (*seeded data*), como a conta `admin@juice-sh.op`, cuja existência é facilmente dedutível e que, conjugada com a ausência de mecanismos de bloqueio, facilita o acesso indevido.

4.4.3 Testing for Weak Lock Out Mechanism: Para avaliar a robustez do mecanismo de bloqueio de conta e cumprir o requisito prático de realizar um ataque de *Fuzzing* ao formulário de autenticação, recorreu-se à ferramenta *Fuzzer* do OWASP ZAP. **Descobertas:** O ataque consistiu na intercepção de um pedido de login e na injeção automatizada de uma lista de palavras-passe (*wordlist*) contra o utilizador `admin@juice-sh.op`. Conforme evidenciado na captura de tráfego, a aplicação processou todas as tentativas automatizadas de forma contínua (devolvendo o código de erro genérico 401 Unauthorized) sem acionar qualquer mecanismo de proteção. A ausência total de bloqueio de conta após múltiplas tentativas falhadas (*Account Lockout*) ou limitação de pedidos (*Rate Limiting*) permitiu que o ataque prosseguisse até a ferramenta identificar a palavra-passe correta (`admin123`), que obteve a resposta 200 OK. Trata-se de uma falha crítica que facilita o comprometimento total de contas através de força bruta.



Task ID	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
0	Original	200 OK		674 ms	386 bytes	799 bytes			
1	Fuzzed	401 Unauthorized		244 ms	387 bytes	26 bytes			12345
2	Fuzzed	401 Unauthorized		792 ms	387 bytes	26 bytes			admin
3	Fuzzed	401 Unauthorized		71 ms	387 bytes	26 bytes			password
4	Fuzzed	200 OK		445 ms	387 bytes	26 bytes	Reflected		admin123
5	Fuzzed	401 Unauthorized		356 ms	387 bytes	26 bytes			

4.4.4 Testing for Bypassing Authentication Schema: Este teste foca-se em identificar formas de contornar completamente o esquema de autenticação, conseguindo acesso sem necessitar de adivinhar credenciais legítimas.

Descobertas: Detetou-se uma vulnerabilidade crítica no formulário de *login*. O campo de correio eletrónico não sanitiza adequadamente os dados introduzidos antes de os processar na base de dados. Ao injetar uma *payload* básica de SQL Injection (ex: `' OR 1=1 --` ou `admin@juice-sh.op' --`) no campo de email,

com qualquer palavra-passe arbitrária, foi possível manipular a consulta SQL no *backend*, contornar a validação e efetuar *login* imediato com privilégios de Administrador. Esta falha compromete totalmente a barreira de autenticação.

4.4.5 Testing for Vulnerable Remember Password: Avaliou-se a forma como a aplicação lida com a persistência de sessões para os utilizadores que fecham o *browser* e regressam mais tarde. **Descobertas:** A aplicação, sendo uma *Single Page Application* (SPA) em Angular, gere a autenticação baseada em *tokens* JWT (JSON Web Tokens). Quando o *login* é efetuado, o *token* de sessão é armazenado em formato de texto limpo no *Local Storage* do *browser* do cliente. Ao contrário dos *cookies* seguros (que podem usar a *flag* *HttpOnly* para impedir a leitura por JavaScript), o *Local Storage* está diretamente acessível a qualquer *script* em execução na página. Isto significa que uma vulnerabilidade de *Cross-Site Scripting* (XSS) noutra secção do site permitiria a um atacante roubar facilmente este *token* e comprometer a sessão do utilizador.

4.4.6 Testing for Browser Cache Weaknesses: O objetivo deste teste é verificar se informações sensíveis ou páginas que requerem autenticação ficam indevidamente guardadas na *cache* local do *browser*. **Descobertas:** Realizou-se um teste de retenção de *cache*: o utilizador efetuou o *login*, acedeu a páginas restritas (como detalhes do perfil ou histórico de encomendas), efetuou o *logout* e, em seguida, utilizou o botão "Retroceder" (*Back*) do *browser*. Verificou-se que as páginas restritas voltaram a ser apresentadas a partir da *cache* local, expondo as informações sensíveis lá contidas a qualquer utilizador que tenha acesso físico ao dispositivo, falhando em forçar o *browser* a requerer uma revalidação (*Cache-Control: no-cache, no-store*).

4.4.7 Testing for Weak Password Policy: Avaliou-se a política de palavras-passe exigida no momento do registo de uma nova conta. **Descobertas:** Verificou-se que a aplicação permite a criação de contas exigindo um comprimento mínimo de apenas 5 caracteres para a palavra-passe, sem impor qualquer complexidade adicional (como a obrigatoriedade de letras maiúsculas, números ou caracteres especiais). Esta política é considerada extremamente permissiva, tornando as credenciais dos utilizadores altamente suscetíveis a ataques de força bruta ou de dicionário.

4.4.8 Testing for Weak Security Question e 4.4.9 Testing for Weak Password Reset: Estas secções avaliam em conjunto a segurança dos mecanismos de recuperação de conta em caso de esquecimento da palavra-passe. **Descobertas:** Constatou-se que o mecanismo de recuperação de conta (funcionalidade "Forgot Password") baseia-se única e exclusivamente na resposta a uma "Pergunta de Segurança" estática, definida pelo utilizador no momento do registo. A aplicação não envia qualquer *link* ou *token* temporário de verificação por um canal secundário seguro (*out-of-band*), como o endereço de correio eletrónico associado. Como as

respostas a perguntas de segurança (ex: nome de familiares ou animais de estimação) são frequentemente fáceis de adivinhar ou recolher através de engenharia social, este mecanismo compromete severamente a resiliência do sistema de autenticação e facilita a apropriação indevida de contas (*Account Takeover*).

4.4.10 Testing for Weaker Authentication in Alternative Channel: O objetivo deste teste é verificar se a aplicação disponibiliza canais alternativos (como interfaces móveis dedicadas, portais antigos/legados ou APIs secundárias) que utilizem mecanismos de autenticação menos seguros do que o canal web principal, permitindo a um atacante contornar os controlos de segurança. **Descobertas:** A aplicação OWASP Juice Shop possui uma arquitetura centralizada baseada em *Single Page Application* (SPA), na qual a interface web e qualquer dispositivo cliente comunicam diretamente com a mesma API RESTful no *backend*. Durante o mapeamento da aplicação, não foram identificados canais alternativos de autenticação paralelos à API principal. Uma vez que a gestão de sessões e o processo de autenticação (baseado em JWT) são unificados e excluídos a este único canal, esta vulnerabilidade não se aplica ao presente cenário.

5 - Authorization Testing

4.5.1 Testing Directory Traversal File Include: O objetivo deste teste é verificar se a aplicação permite a um atacante aceder a ficheiros ou diretórios não autorizados no servidor web, contornando os controlos de acesso. **Descobertas:** Ao explorar o diretório exposto /ftp (identificado na fase de *Information Gathering*), tentou-se descarregar ficheiros sensíveis como `package.json.bak`. A aplicação apresentou um erro, revelando que existe um filtro que apenas permite o descarregamento de ficheiros com as extensões `.md` ou `.pdf`. No entanto, este mecanismo de validação é suscetível a técnicas de evasão. Um atacante pode utilizar a técnica de *Poison Null Byte* (anexando `%00.pdf` ao nome do ficheiro, ex: `package.json.bak%00.pdf`) para enganar o filtro de extensão do *frontend* e forçar o *backend* a ignorar o resto da *string*, resultando no descarregamento não autorizado de ficheiros de sistema e configuração (*Directory Traversal / Arbitrary File Download*).

4.5.2 Testing for Bypassing Authorization Schema: Este teste foca-se em avaliar se é possível contornar o esquema de autorização para aceder a recursos destinados a outros utilizadores. **Descobertas:** Durante a análise da API RESTful da aplicação, verificou-se que o controlo de acessos a determinados recursos falha em validar a propriedade do objeto solicitado face ao utilizador autenticado. Isto ficou comprovado pela manipulação direta dos pedidos referentes ao cesto de compras, vulnerabilidade que é detalhada em profundidade na secção 4.5.4.

4.5.3 Testing for Privilege Escalation: O objetivo deste teste é verificar se um utilizador consegue escalar os seus privilégios (verticalmente) para obter acessos de administrador, ou (horizontalmente) para aceder a contas do mesmo nível. **Descobertas:** Verificou-se que a aplicação é vulnerável a escalonamento vertical de privilégios. Através da exploração da vulnerabilidade de Injeção SQL no formulário de autenticação (documentada na secção 4.4.4), um atacante consegue forçar o *login* na conta `admin@juice-sh.op`. Uma vez autenticado com esta conta, o esquema de autorização baseia-se integralmente no *token* JWT atribuído, o que permite o acesso direto e não restrito ao painel de administração (`/#/administration`), que de outra forma estaria vedado (Erro 403) a utilizadores comuns.

4.5.4 Testing for Insecure Direct Object References (IDOR): Este teste avalia se a aplicação expõe referências diretas a objetos internos (como identificadores na base de dados) sem a devida validação de autorização. **Descobertas:** Detetou-se uma vulnerabilidade crítica de IDOR no mecanismo de gestão do cesto de compras (*Basket*). Quando um utilizador acede ao seu cesto, a aplicação efetua um pedido HTTP GET à API utilizando um identificador numérico sequencial (ex: `/rest/basket/2`). Ao utilizar o OWASP ZAP para interceptar o tráfego e alterar manualmente este identificador para outro valor (ex: `/rest/basket/1`), o servidor responde com sucesso, devolvendo o conteúdo do cesto de compras pertencente a outro utilizador. A falta de validação de autorização no *backend* permite que qualquer utilizador autenticado visualize e manipule os objetos de outros utilizadores.

6 - Session Management Testing

4.6.1 Testing for Session Management Schema: O objetivo deste teste é compreender como a aplicação gere o estado e a identificação do utilizador após a autenticação. **Descobertas:** A aplicação, sendo uma *Single Page Application* (SPA), não utiliza o mecanismo tradicional de sessões mantidas no servidor (*Server-side Sessions*). Em vez disso, utiliza um esquema *stateless* baseado em **JSON Web Tokens (JWT)**. Após o *login* com sucesso, a API gera um JWT e envia-o para o cliente. O *browser* guarda este *token* no *Local Storage* e inclui-o no cabeçalho HTTP (`Authorization: Bearer <token>`) em cada pedido subsequente à API.

4.6.2 Testing for Cookies Attributes: Avaliamos a segurança dos *cookies* emitidos pelo servidor, focando nas *flags* de proteção. **Descobertas:** Embora a sessão principal utilize JWT, a aplicação gere outras preferências através de *cookies*. Verificou-se que os atributos de segurança vitais, como `Secure` e `HttpOnly`, estão ausentes. Como a aplicação opera em ambiente local sobre o protocolo HTTP, a *flag* `Secure` (que obriga o envio do cookie apenas por HTTPS) nunca é ativada, deixando a informação suscetível a interceção na rede.

4.6.3 Testing for Session Fixation: Este teste verifica se é possível forçar um utilizador a autenticar-se utilizando um identificador de sessão previamente conhecido pelo atacante. **Descobertas:** A arquitetura baseada em JWT mitiga os ataques clássicos de Fixação de Sessão. Após cada autenticação bem-sucedida, o servidor gera dinamicamente um novo *token* assinado digitalmente, não aceitando nem reaproveitando identificadores ou *tokens* fornecidos pelo cliente antes do *login*.

4.6.4 Testing for Exposed Session Variables: O objetivo é identificar se identificadores de sessão são expostos em locais inseguros, como o URL.

Descobertas: Conforme detetado previamente pelo varrimento automático do OWASP ZAP (que gerou o alerta de "**Session ID in URL Rewrite**" documentado na secção 1), a aplicação expõe identificadores de sessão em parâmetros do URL em determinadas circunstâncias. Adicionalmente, o armazenamento do JWT no *Local Storage* (em vez de um *cookie HttpOnly*) expõe a principal variável de sessão à leitura por *scripts* do lado do cliente, o que se torna um risco crítico em caso de vulnerabilidades XSS.

4.6.5 Testing for Cross Site Request Forgery (CSRF): Avaliou-se a suscetibilidade da aplicação à falsificação de pedidos entre *sites*. **Descobertas:** O esquema de gestão de sessões utilizado constitui, por defeito, uma defesa robusta contra ataques CSRF. Como a autenticação depende do cabeçalho *Authorization: Bearer*, e os *browsers* não anexam automaticamente cabeçalhos personalizados em pedidos cruzados (ao contrário do que fazem com os *cookies* de sessão), a exploração de CSRF torna-se ineficaz na grande maioria dos *endpoints* da API do Juice Shop.

4.6.6 Testing for Logout Functionality e 4.6.7 Testing Session Timeout: Estas duas secções foram avaliadas em conjunto devido à natureza da arquitetura *stateless*. **Descobertas:** O mecanismo de término de sessão (*Logout*) e de expiração apresenta falhas conceptuais graves inerentes à má implementação do JWT. Quando o utilizador clica em "Logout", a interface *frontend* apaga simplesmente o *token* do *Local Storage*. No entanto, o *token não é invalidado no servidor* (não existe uma *blocklist* ou revogação). Se um atacante intercetar o JWT, poderá continuar a utilizá-lo para aceder à conta de forma ilegítima mesmo após o utilizador ter efetuado o *logout*, permanecendo válido até que o tempo de vida intrínseco (Timeout/Expiração) definido no *payload* do *token* seja atingido.

4.6.8 Testing for Session Puzzling: Este teste procura anomalias onde variáveis de sessão são sobrepostas ou mal interpretadas pelo servidor. **Descobertas:** Durante os testes realizados aos *endpoints* da API, não foram identificadas vulnerabilidades evidentes de confusão de sessão (*Session Overloading/Puzzling*), dado que o estado do utilizador e os seus privilégios estão encapsulados de forma rígida dentro do *payload* codificado do JSON Web Token, cuja integridade é validada pela assinatura do servidor.

4.6.9 Testing for Session Hijacking: Avaliou-se o risco geral de um atacante conseguir sequestrar uma sessão legítima. **Descobertas:** A aplicação é avaliada como altamente vulnerável ao Sequestro de Sessão (*Session Hijacking*). A conjugação de múltiplos fatores arquitetónicos já documentados (a ausência de encriptação TLS/SSL em trânsito, a exposição do *token* no *Local Storage*, e a ausência de um mecanismo efetivo de revogação de *tokens* após o *logout*) fornece a um atacante diversas vias fiáveis (seja via *sniffing* de rede ou exploração de *Cross-Site Scripting*) para roubar o JWT e assumir o controlo integral e prolongado da sessão do utilizador.

7- Input Validation Testing

4.7.1 Testing for Reflected Cross Site Scripting (XSS): O objetivo deste teste é verificar se a aplicação reflete dados não sanitizados diretamente na resposta HTTP. **Descobertas:** Através de testes manuais e varrimentos do OWASP ZAP, identificou-se uma vulnerabilidade de XSS Refletido na barra de pesquisa da aplicação. Ao injetar uma *payload* em JavaScript (ex: `<iframe src="javascript:alert(xss)">`) no parâmetro de pesquisa (q), o servidor devolve esse mesmo código embutido no HTML da página de resultados. Como o *browser* confia no código recebido, executa a *payload* imediatamente. Esta falha permite a execução de *scripts* no contexto da sessão da vítima caso esta clique num *link* malicioso preparado pelo atacante.

4.7.2 Testing for Stored Cross Site Scripting (XSS): Este teste avalia se a aplicação armazena *inputs* maliciosos na base de dados e os apresenta posteriormente a outros utilizadores sem a devida higienização. **Descobertas:** Detetou-se uma falha crítica de XSS Armazenado na funcionalidade de "Customer Feedback" (Reclamações/Sugestões). A aplicação aceita submissões contendo *tags* HTML e *scripts* JavaScript no campo de comentário. Estes dados são guardados na base de dados. Quando um utilizador com privilégios de administração acede ao painel de gestão (`/#/administration`) para ler o *feedback*, o código injetado é processado e executado automaticamente no seu *browser*. Isto pode ser explorado para roubar o *token* JWT do administrador ou forçar ações indesejadas em seu nome.

4.7.3 Testing for HTTP Verb Tampering e 4.7.4 Testing for HTTP Parameter Pollution: **Descobertas:** Durante a interceção de tráfego no ZAP, manipularam-se os métodos HTTP e injetaram-se parâmetros duplicados nas chamadas à API REST. Embora não se tenham verificado desvios críticos na lógica de negócio através da poluição de parâmetros, verificou-se que a API responde de forma inconsistente a verbos não padronizados, sublinhando a necessidade de uma política de rejeição estrita (Permitir apenas GET/POST/PUT/DELETE onde aplicável).

4.7.5 Testing for SQL Injection: O objetivo é detetar se os dados introduzidos pelos utilizadores podem alterar a estrutura das consultas (*queries*) executadas na base de dados relacional. **Descobertas:** Conforme já referenciado na secção de Autenticação (4.4.4), a aplicação é criticamente vulnerável a Injeção SQL. O campo de correio eletrónico no formulário de *login* concatena diretamente o texto do utilizador na consulta SQL do *backend*. Injetando o valor `admin@juice-sh.op' --`, a aspa simples fecha a *string* esperada e os traços comentam a validação da palavra-passe. O *backend*, que utiliza **SQLite**, executa a consulta alterada, resultando no contorno total da autenticação.

4.7.5.6 Testing for NoSQL Injection: A avaliação procurou falhas de injeção em componentes que utilizem bases de dados não relacionais. **Descobertas:** A arquitetura do Juice Shop utiliza bases de dados NoSQL (como o MongoDB) para certas funcionalidades, como as avaliações de produtos (*Product Reviews*). Manipulando os pedidos da API (como o envio de objetos JSON inesperados { `"$ne": null` } em vez de *strings* simples), foi possível influenciar a lógica da base de dados, permitindo a edição ou manipulação de avaliações alheias, confirmando a vulnerabilidade a NoSQL Injection.

4.7.11 Testing for Code Injection e 4.7.11.1 Testing for Local File Inclusion

(LFI): Estes testes procuram identificar falhas que permitam a execução de código arbitrário ou a inclusão de ficheiros locais do servidor. **Descobertas:** Em alinhamento com os testes de *Directory Traversal* detalhados na secção 4.5.1, confirmou-se uma vulnerabilidade de LFI. A funcionalidade responsável por servir ficheiros de idioma ou documentos no diretório `/ftp` não restringe adequadamente o caminho procurado. Utilizando técnicas de navegação de diretórios (`.././../`), um atacante consegue forçar a aplicação a ler e apresentar ficheiros internos do sistema operativo e configurações de código-fonte.

4.7.19 Testing for Server-Side Request Forgery (SSRF): Avaliou-se se a aplicação pode ser manipulada para efetuar pedidos HTTP arbitrários a partir do servidor *backend*. **Descobertas:** A aplicação possui vulnerabilidades de SSRF em áreas que requerem a obtenção de recursos remotos, permitindo a um atacante induzir o servidor a realizar pedidos à sua própria rede interna (`localhost`) ou a redes privadas inatingíveis pelo exterior, expondo portas e serviços internos.

Secções Não Aplicáveis (Out of Scope)

As seguintes secções de testes de validação de dados são consideradas fora do âmbito desta análise. A exclusão justifica-se pela arquitetura tecnológica identificada no servidor alvo (Node.js, Express, Angular, SQLite), o que impossibilita a existência destas vulnerabilidades específicas na infraestrutura:

- **4.7.5.1 a 4.7.5.5 (Injeções em Oracle, MySQL, SQL Server, PostgreSQL, MS Access):** A aplicação utiliza predominantemente SQLite, não possuindo motores destas bases de dados.
- **4.7.6 Testing for LDAP Injection:** Não existe integração com serviços de diretório (Active Directory/LDAP).
- **4.7.7 a 4.7.10 (XML, SSI, XPath, IMAP/SMTP Injection):** A aplicação processa dados quase exclusivamente em formato JSON através de API REST, não recorrendo a parsers XML, XPath ou integração direta de correio com manipulação de *headers* SMTP.
- **4.7.13 Testing for Format String Injection:** Vulnerabilidade associada a linguagens C/C++, inexistente neste ambiente Node.js.
- **4.7.11.2 (RFI), 4.7.12 (Command Injection), 4.7.14 a 4.7.18:** Foram excluídos por restrições de tempo de laboratório ou por não representarem o vetor de ataque primário detetado nas análises do OWASP ZAP neste âmbito

8 - Testing for Error Handling

4.8.1 Testing for Improper Error Handling: O objetivo deste teste é verificar se a aplicação lida com situações de erro de forma segura, apresentando mensagens genéricas aos utilizadores sem revelar informações sensíveis sobre a infraestrutura ou a lógica de negócio. **Descobertas:** Durante a interação com a API REST, verificou-se que a submissão de pedidos malformados (como *payloads* JSON com sintaxe inválida ou manipulação de parâmetros com tipos de dados inesperados) resulta em mensagens de erro excessivamente verbosas. Em vez de intercepar os erros de forma centralizada e devolver uma resposta genérica (ex: "Ocorreu um erro inesperado"), a aplicação devolve detalhes específicos sobre a falha na validação dos dados, expondo a lógica interna do *backend* e facilitando a afinação de ataques.

4.8.2 Testing for Stack Traces: Este teste procura determinar se exceções não tratadas na aplicação resultam na devolução de rastreios de pilha (*stack traces*) diretamente ao cliente web. **Descobertas:** Ao provocar erros críticos na aplicação (por exemplo, forçando erros de sintaxe nas consultas durante as tentativas iniciais de Injeção SQL ou submetendo *tokens* JWT com cabeçalhos corrompidos), confirmou-se que a framework baseada em Express/Node.js falha em capturar e tratar a exceção de forma segura. Como resultado, o servidor devolve um *stack trace* completo no corpo da resposta HTTP. Esta falha expõe o fluxo de execução do código, caminhos absolutos de ficheiros locais no servidor e as versões das bibliotecas em uso, fornecendo dados de reconhecimento inestimáveis para a preparação de vetores de ataque mais complexos.

9 - Testing for Weak Cryptography:

Esta secção não foi avaliada porque, de acordo com as diretrizes do projeto, as comunicações com a aplicação Juice Shop no ambiente de laboratório não são efetuadas sobre HTTPS.

10 - Business Logic Testing:

Os testes à lógica de negócio da aplicação não foram considerados para este relatório, uma vez que foram explicitamente definidos como estando fora do âmbito deste trabalho prático.

11 - Client-side Testing

4.11.1 Testing for DOM-Based Cross Site Scripting (XSS) e 4.11.2 Testing for JavaScript Execution: O objetivo destes testes é verificar se a aplicação processa dados perigosos (*sources*) e os reflete no Document Object Model (DOM) de forma insegura (*sinks*), permitindo a execução de código no cliente. **Descobertas:**

Tratando-se de uma aplicação Angular, a interface depende fortemente da manipulação dinâmica do DOM baseada em interações do utilizador e parâmetros de URL. Confirmou-se a presença de DOM XSS em funcionalidades de pesquisa e no processamento de parâmetros de idioma (`?language=`). Ao injetar *payloads* em JavaScript, a arquitetura do *frontend* interpreta e executa o código diretamente no *browser* da vítima sem que a *payload* necessite de ser previamente enviada ou validada pelo servidor, contornando potenciais defesas baseadas em rede (WAF).

4.11.3 Testing for HTML Injection: Este teste avalia se um atacante consegue injetar marcação HTML arbitrária na interface. **Descobertas:** Verificou-se que áreas da aplicação que refletem perfis de utilizador ou mensagens não codificam corretamente as entidades HTML (`<`, `>`, `&`). Isto permitiu a injeção de *tags* estruturais (como `<h1>` ou `<img>`), alterando o aspeto visual da página (*defacement*). Embora menos grave que o XSS (pois não executa *scripts* ativos), a injeção de HTML pode ser utilizada para desenhar formulários falsos (*phishing*) dentro da página legítima para roubar credenciais.

4.11.4 Testing for Client-side URL Redirect: O objetivo é identificar se a aplicação permite o redirecionamento forçado do *browser* para domínios arbitrários não controlados. **Descobertas:** A aplicação possui uma funcionalidade vulnerável a *Open Redirect* (Redirecionamento Aberto) no lado do cliente. Identificaram-se *endpoints* e hiperligações (como o reencaminhamento para o GitHub do projeto) que utilizam parâmetros de URL (ex: `/redirect?to=`) para determinar o destino. Como a aplicação não valida o destino fornecido (*allowlist*), um atacante pode criar um *link* do Juice Shop que encaminha silenciosamente a vítima para uma página maliciosa de *phishing*, explorando a confiança do utilizador no domínio original.

4.11.9 Testing for Clickjacking: Avaliou-se a resistência da aplicação a ataques de sobreposição de interface (*UI Redressing*), onde a página legítima é embutida num

iframe invisível. **Descobertas:** A análise aos cabeçalhos HTTP interceptados pelo OWASP ZAP (previamente documentada na Fase 1 de *Configuration Testing*) revelou a ausência de mecanismos de defesa contra Clickjacking. O servidor não emite o cabeçalho X-Frame-Options: DENY/SAMEORIGIN, nem a diretiva Content-Security-Policy: frame-ancestors. Consequentemente, um atacante pode embutir o Juice Shop numa página externa maliciosa e induzir a vítima a clicar em botões invisíveis, executando ações indesejadas (como transferências de fundos ou apagar a conta) enquanto a vítima acredita estar a clicar num conteúdo inofensivo.

4.11.12 Testing Browser Storage: O objetivo é verificar se informações sensíveis são armazenadas de forma insegura nas tecnologias de armazenamento local do *browser* (HTML5 Storage). **Descobertas:** Conforme amplamente discutido na secção de Gestão de Sessões (4.6.1 e 4.6.4), a aplicação armazena de forma persistente o *token* de autenticação JWT (JSON Web Token) no Local Storage do *browser*, completamente em texto limpo. Uma vez que os dados no Local Storage não expiram ao fechar o *browser* e estão totalmente acessíveis através do objeto global de JavaScript (`window.localStorage`), o roubo de sessões torna-se trivial na presença de qualquer vulnerabilidade de XSS na aplicação.

Secções Não Aplicáveis ou Fora de Âmbito

De acordo com a arquitetura analisada e as limitações do ambiente de laboratório, os seguintes testes cliente-servidor não foram considerados vetores de ataque primários ou aplicáveis:

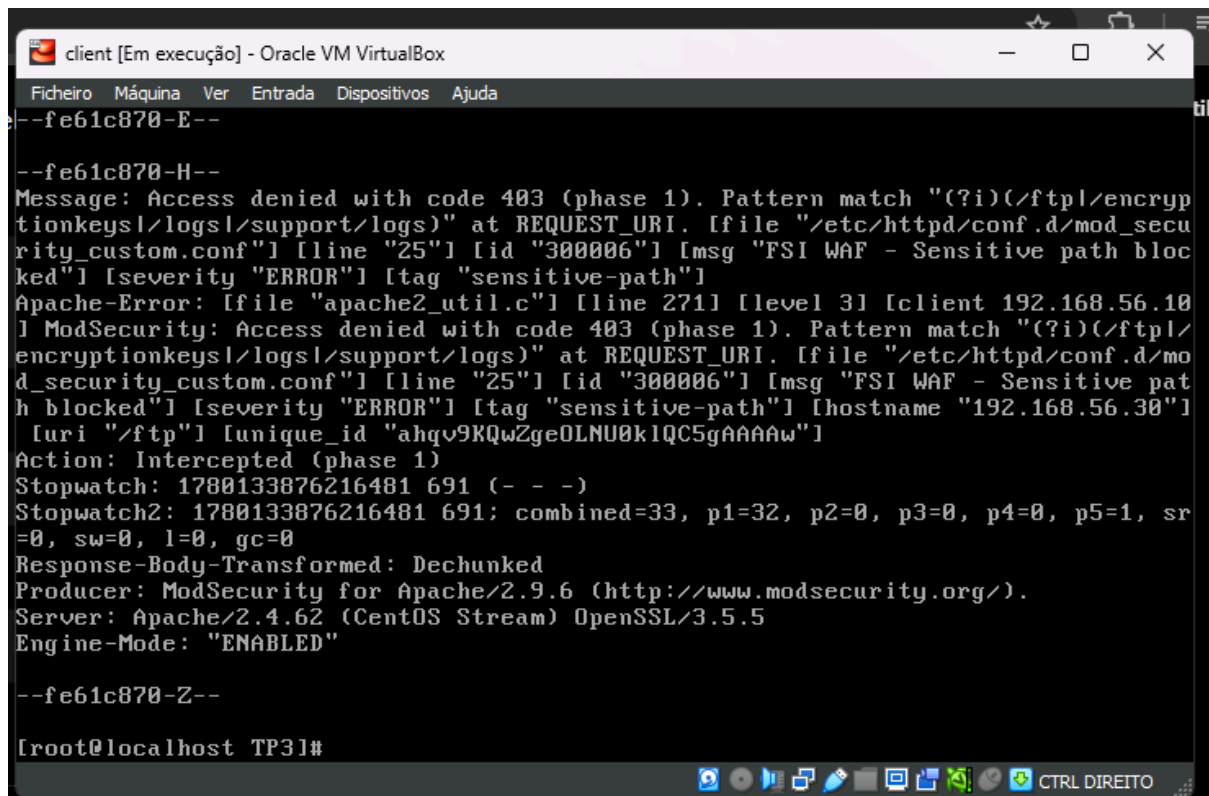
- **4.11.5 Testing for CSS Injection e 4.11.6 Testing for Client-side Resource Manipulation:** Não representam um risco estrutural primário comparado ao XSS no contexto atual da aplicação.
- **4.11.7 Testing Cross Origin Resource Sharing (CORS):** As más configurações de domínio cruzado foram testadas e documentadas no capítulo de Configuração (Secção 4.2.8).
- **4.11.8 Testing for Cross Site Flashing:** Tecnologia completamente obsoleta e ausente da aplicação (Adobe Flash Player não é suportado por *browsers* modernos).
- **4.11.10 Testing WebSockets e 4.11.11 Testing Web Messaging:** A comunicação principal assenta em pedidos assíncronos HTTP (AJAX/XHR) padrão com a API REST, não havendo dependência de *WebSockets* ou canais complexos de *PostMessage* para a lógica de negócio principal.
- **4.11.13 Testing for Cross Site Script Inclusion:** Fora do âmbito principal focado nas mecânicas diretas da SPA.

4)Web application security firewall

Nesta segunda fase, o ambiente transitou para o Cenário 2, onde o servidor Apache atua como *Reverse Proxy* (configurado em `juice_shop_proxy.conf`) e integra o ModSecurity como Web Application Firewall (WAF). As seguintes secções descrevem como as regras implementadas no ficheiro `modsecurity.conf` mitigarão as vulnerabilidades específicas identificadas na Fase 1.

4.1 Information Gathering Na Fase 1, ferramentas como o Nmap foram utilizadas para realizar o varrimento do servidor (Secção 4.1.2). Com a WAF ativa, a **Regra 300005** atua na Fase 1 do ModSecurity para detetar e bloquear *User-Agents* associados a scanners conhecidos (como nmap, sqlmap, dirbuster, entre outros), mitigando o reconhecimento automatizado. Adicionalmente, a exposição indevida do diretório oculto `/ftp` detetada através do ficheiro `robots.txt` (Secção 4.1.3) foi retificada pela **Regra 300006**, que bloqueia explicitamente o acesso direto a caminhos sensíveis como `/ftp`, `/logs` e `/encryptionkeys`.

```
(root@kasckepard)-[~/FSI/FSI/TP3]
# curl "http://192.168.56.30/ftp"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
</body></html>
```



```
client [Em execução] - Oracle VM VirtualBox
Ficheiro Máquina Ver Entrada Dispositivos Ajuda
--fe61c870-E--
--fe61c870-H--
Message: Access denied with code 403 (phase 1). Pattern match "(?i)(/ftp/encryptionkeys/logs/support/logs)" at REQUEST_URI. [file "/etc/httpd/conf.d/mod_security_custom.conf"] [line "25"] [id "300006"] [msg "FSI WAF - Sensitive path blocked"] [severity "ERROR"] [tag "sensitive-path"]
Apache-Error: [file "apache2_util.c"] [line 271] [level 3] [client 192.168.56.10] ModSecurity: Access denied with code 403 (phase 1). Pattern match "(?i)(/ftp/encryptionkeys/logs/support/logs)" at REQUEST_URI. [file "/etc/httpd/conf.d/mod_security_custom.conf"] [line "25"] [id "300006"] [msg "FSI WAF - Sensitive path blocked"] [severity "ERROR"] [tag "sensitive-path"] [hostname "192.168.56.30"] [uri "/ftp"] [unique_id "ahqv9KQwZgeOLNU0k1QC5gAAAw"]
Action: Intercepted (phase 1)
Stopwatch: 1780133876216481 691 (- - -)
Stopwatch2: 1780133876216481 691; combined=33, p1=32, p2=0, p3=0, p4=0, p5=1, sr=0, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.6 (http://www.modsecurity.org/).
Server: Apache/2.4.62 (CentOS Stream) OpenSSL/3.5.5
Engine-Mode: "ENABLED"
--fe61c870-Z--
[root@localhost TP3]#
```

(modsec_audit.log)

4.2 Configuration and Deployment Management Testing A análise inicial revelou que o servidor permitia o descarregamento de extensões sensíveis (Secção 4.2.3) e a exposição de ficheiros antigos e backups dentro do diretório /ftp (Secção 4.2.4). Graças à implementação da **Regra 300006** da WAF, todo o tráfego HTTP direcionado ao caminho /ftp é agora barrado com o status 403, resolvendo o problema de exposição destes backups na raiz.

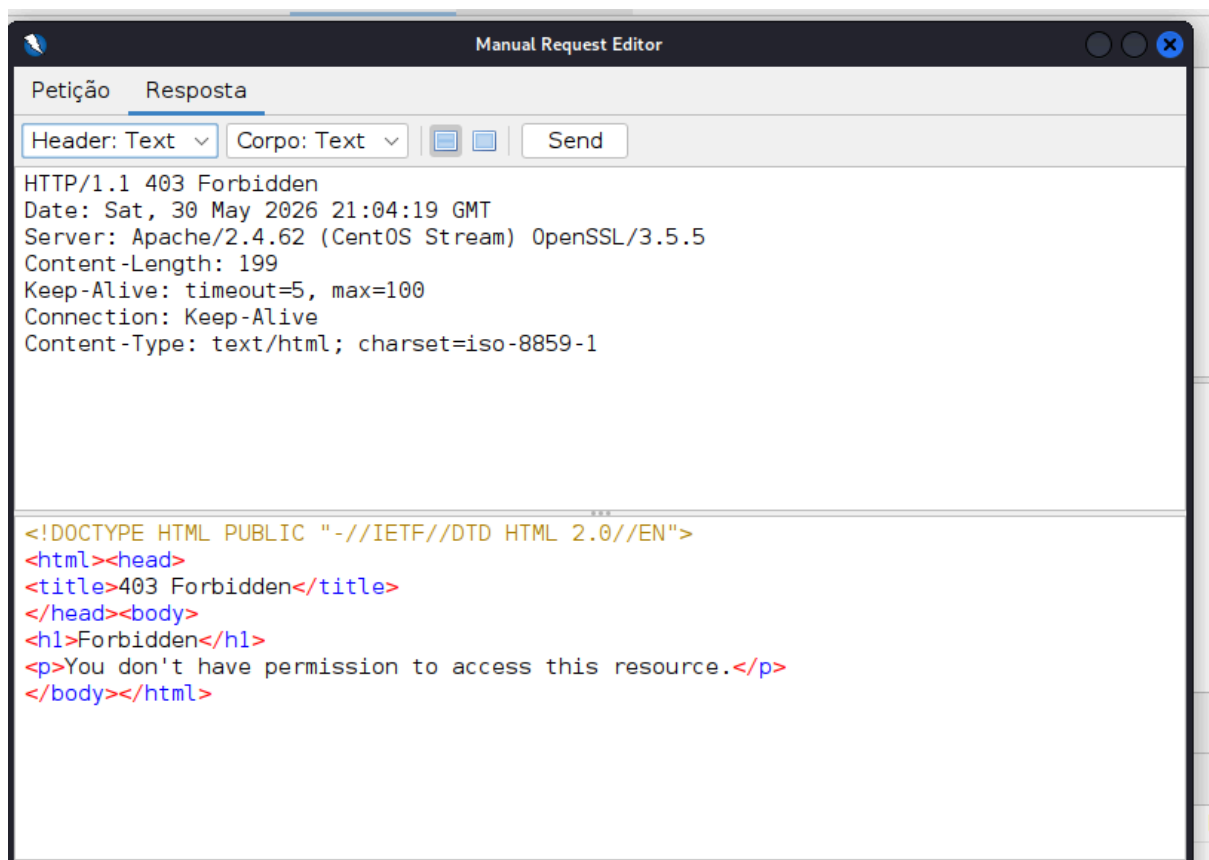
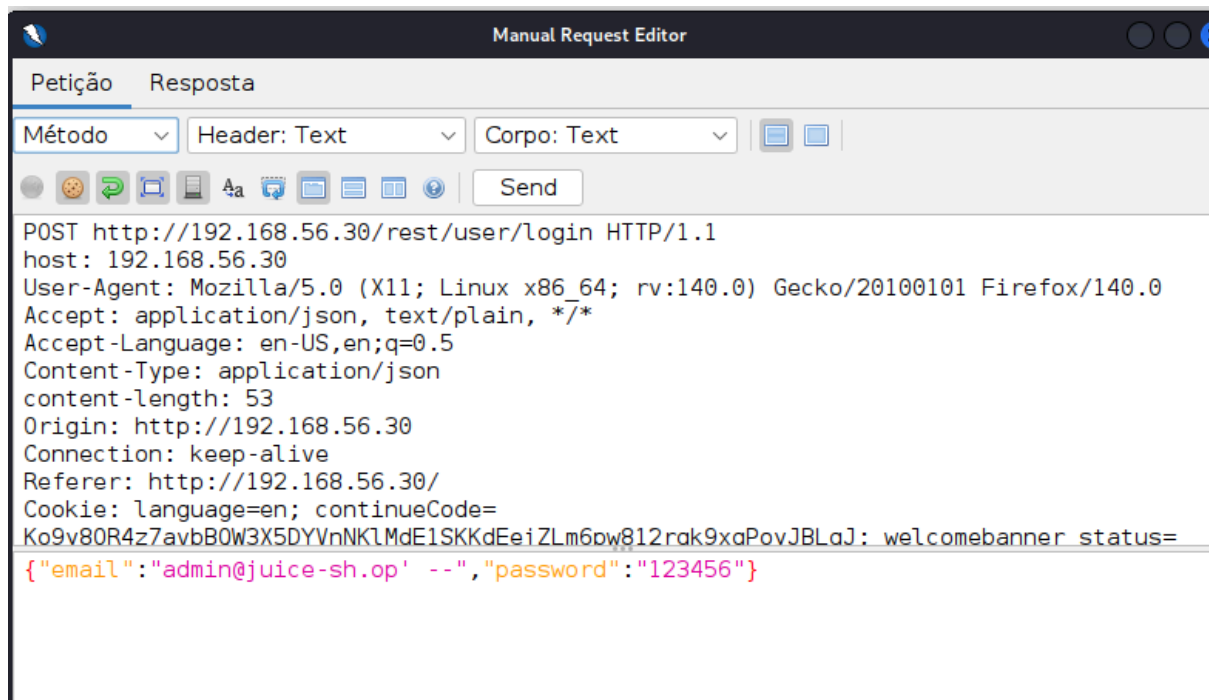
```
# Rule 300006 - Block access to sensitive Juice Shop paths
SecRule REQUEST_URI "@rx (?i)(/ftp/encryptionkeys/logs/support/logs)" \
    "id:300006,phase:1,deny,status:403,log,msg:'FSI WAF - Sensitive path blocked',severity:'ERROR',tag:'sensitive-path'"

```

4.3 Identity Management Testing Conforme indicado nas diretrizes do Guião do Trabalho Prático 3, as falhas de Gestão de Identidade identificadas na Fase 1 (tais como a Enumeração de Contas detalhada na Secção 4.3.4, onde a aplicação devolvia "Email must be unique") tratam-se de vulnerabilidades na lógica da aplicação. Como o ModSecurity opera ao nível do HTTP analisando padrões, estas falhas lógicas não podem ser mitigadas ou resolvidas apenas através da WAF, não requerendo intervenção.

4.4 Authentication Testing A vulnerabilidade mais crítica identificada na autenticação foi o *Bypass* através de Injeção SQL no campo de correio eletrónico (Secção 4.4.4, com a payload admin@juice-sh.op' --). Esta falha é totalmente

erradicada através da **Regra 300001**. A WAF inspeciona os argumentos do pedido e interceta metacaracteres e expressões SQL (--, OR 1=1, union select), bloqueando os ataques e protegendo o mecanismo de login.




```
--7405554c-H--
Message: Access denied with code 403 (phase 2). Operator GT matched 10 at IP:login_counter. [file "/etc/httpd/conf.d/mod_security_custom.conf"] [line "70"] [id "300009"] [msg "FSI WAF - Possible login brute-force/fuzzing attack blocked"] [severity "CRITICAL"] [tag "bruteforce-login"]
Apache-Error: [file "apache2_util.c"] [line 271] [level 3] [client 192.168.56.1] [mod_security: Access denied with code 403 (phase 2). Operator GT matched 10 at IP:login_counter. [file "/etc/httpd/conf.d/mod_security_custom.conf"] [line "70"] [id "300009"] [msg "FSI WAF - Possible login brute-force/fuzzing attack blocked"] [severity "CRITICAL"] [tag "bruteforce-login"] [hostname "192.168.56.30"] [uri "/socket.io/"] [unique_id "ahtck4TGX4Xpxvxbx3MwBwAAAAIA"]
Action: Intercepted (phase 2)
Apache-Handler: proxy-server
Stopwatch: 1780178067922189 577 (- - -)
Stopwatch2: 1780178067922189 577; combined=174, p1=133, p2=40, p3=0, p4=0, p5=1, sr=95, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.6 (http://www.modsecurity.org/).
Server: Apache/2.4.62 (CentOS Stream) OpenSSL/3.5.5
Engine-Mode: "ENABLED"

--7405554c-Z--
```

The screenshot shows the Burp Suite interface. The 'Sites' list on the left includes 'Contextos', 'Contexto Padrão', and 'Sites'. The central pane displays the response details for a 403 Forbidden status. The response headers include: HTTP/1.1 403 Forbidden, Date: Sat, 30 May 2026 22:09:31 GMT, Server: Apache/2.4.62 (CentOS Stream) OpenSSL/3.5.5, Content-Length: 199, Keep-Alive: timeout=5, max=100, Connection: Keep-Alive, Content-Type: text/html; charset=iso-8859-1. The response body contains the following HTML:

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
</body></html>
```

The bottom pane shows a table of request history with columns: ID, Source, Req. Timestamp, Método, URL, Code, Reason, RTT, Size Resp. Body, and Highest Alert. The table contains 10 rows of data, all showing a 403 Forbidden status.

ID	Source	Req. Timestamp	Método	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert
46787	Proxy	31/05/26, 19:41:35	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	6 ms	199 bytes	
46788	Proxy	31/05/26, 19:41:40	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	10 ms	199 bytes	
46789	Proxy	31/05/26, 19:41:45	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	4 ms	199 bytes	
46790	Proxy	31/05/26, 19:41:50	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	4 ms	199 bytes	
46791	Proxy	31/05/26, 19:41:56	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	42 ms	199 bytes	
46792	Proxy	31/05/26, 19:42:01	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	8 ms	199 bytes	
46793	Proxy	31/05/26, 19:42:06	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	6 ms	199 bytes	
46794	Proxy	31/05/26, 19:42:11	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	5 ms	199 bytes	
46795	Proxy	31/05/26, 19:42:16	GET	http://192.168.56.30/socket.io/?EIO=4&transport...	403	Forbidden	32 ms	199 bytes	

4.5 Authorization Testing Durante os testes de autorização, foi possível recorrer a *Poison Null Byte* e *Directory Traversal* para aceder e descarregar ficheiros sensíveis do sistema operativo como `package.json.bak` no diretório FTP (Secção 4.5.1). A **Regra 300003** foi desenhada especificamente para este fim, bloqueando tentativas de navegação de diretórios e a submissão de padrões de *Path Traversal* (como `../` ou `%2e%2e%2f`), garantindo que o acesso é recusado antes de atingir o servidor [Node.js](#).

Adicionalmente, embora não ativamente explorado com sucesso massivo na Fase 1, a WAF inclui de forma proativa a **Regra 300004**, garantindo que ataques futuros de execução remota de comandos (RCE) no ambiente de *backend* (Linux) sejam impedidos caso novos vetores de injeção sejam descobertos.

4.8 Testing for Error Handling A exposição excessiva de *Stack Traces* (Secção 4.8.2) resultava de exceções não tratadas quando eram submetidas, por exemplo, consultas de sintaxe maliciosa para testar SQLi. Com as regras do ModSecurity implementadas, os *payloads* maliciosos são imediatamente filtrados e o tráfego bloqueado na camada *Proxy*. Assim, a aplicação Node.js nunca chega a processar estes pedidos inválidos, prevenindo totalmente a geração e a consequente fuga destas informações verbosas de depuração.

4.9 Testing for Weak Cryptography Conforme estabelecido na Fase 1, esta secção é considerada fora de âmbito, visto que as ligações no laboratório não decorrem sobre HTTPS.

4.10 Business Logic Testing Secção considerada fora de âmbito do presente trabalho prático, conforme indicado nas diretrizes e relatório anterior.

4.11 Client Side Testing O *DOM-Based XSS* documentado no processamento de parâmetros de idioma (Secção 4.11.1) e a vulnerabilidade de *HTML Injection* estrutural (Secção 4.11.3) sofrem forte mitigação pela camada de proxy reverso. A aplicação da **Regra 300002** impede que *payloads* JavaScript ativos e *tags* de manipulação do DOM sejam processados pelo servidor alvo, protegendo o cliente independentemente das lacunas sanitárias implementadas na framework *Angular* no *frontend*.

5) Conclusões

A primeira fase do Trabalho Prático possibilitou uma exploração abrangente do ambiente aplicacional do OWASP Juice Shop, provando, através do WSTG, que a ausência de sanitização expõe a infraestrutura a perigos graves. Na segunda fase, a implementação do servidor Web Apache configurado como *Reverse Proxy* transparente, aliado ao ModSecurity, evidenciou a capacidade defensiva de uma *Web Application Firewall*. A aplicação das regras customizadas para mitigar tráfego (com foco em scanners, ficheiros expostos, RCE, Path Traversal, XSS e SQLi) neutralizou os principais vetores de ataque detetados. Fica demonstrado, desta forma, que a integração combinada de análise de vulnerabilidades com a adoção de WAFs como camada compensatória (*Defense-in-Depth*) é vital para blindar os ativos web contra ataques na camada aplicacional antes de eventuais correções a nível do código-fonte.

